

基于多 Agent 的 C2Rust 代码翻译
软件需求规约

版本 <1.2>

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

修订历史记录

日期	版本	说明	作者
12/11/2024	1.0	简介、整体说明、非功能需求、其他产品需求	谢琛
12/11/2024	1.1	接口需求	王超凡
12/11/2024	1.2	功能需求	尹喆

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

目录

1.	简介	5
1.1	目的	5
1.2	定义、首字母缩写词和缩略语	5
1.3	参考资料	5
2.	整体说明	5
2.1	产品总体效果	5
2.2	产品功能	6
2.3	用户特征	6
2.4	约束	6
2.5	假设与依赖关系	6
2.6	需求子集	6
3.	功能需求	7
3.1	Use case 图	7
3.2	Use case1: 转换 C 代码-规约	7
3.2.1	概述	7
3.2.2	参与者	7
3.2.3	前置条件	7
3.2.4	事件流	7
3.2.5	后置条件	7
3.2.6	特殊需求	8
3.3	Use case2: 优化代码-规约	8
3.3.1	概述	8
3.3.2	参与者	8
3.3.3	前置条件	8
3.3.4	事件流	8
3.3.5	后置条件	8
3.3.6	特殊需求	8
3.4	Use case3: 处理静态分析-规约	8
3.4.1	概述	8
3.4.2	参与者	8
3.4.3	前置条件	8
3.4.4	事件流	8
3.4.5	后置条件	9
3.4.6	特殊需求	9
3.5	Use case4: 运行测试-规约	9
3.5.1	概述	9
3.5.2	参与者	9
3.5.3	前置条件	9
3.5.4	事件流	9
3.5.5	后置条件	9
3.5.6	特殊需求	9
4.	非功能需求	9
4.1	易用性	9

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

4.2	可靠性	10
4.2.1	错误界定	10
4.2.2	可靠性需求	10
4.3	性能	10
4.4	可支持性	10
4.4.1	编码标准与命名约定	10
4.4.2	库和框架	10
4.4.3	维护访问权	10
4.4.4	维护实用程序	10
4.5	设计约束	10
4.5.1	软件语言与开发环境	10
4.5.2	软件流程需求	10
4.5.3	开发工具	10
4.5.4	框架及设计约束	10
5.	其它产品需求	11
5.1	接口需求	11
5.1.1	用户界面	11
5.1.2	硬件接口	11
5.1.3	软件接口	12
5.1.4	通信接口	12
5.2	适用的标准	12
5.2.1	法律法规	12
5.2.2	质量标准	12
5.2.3	易用性标准	13
5.2.4	互操作性标准	13
5.2.5	国际化标准	13
5.2.6	安全性标准	13
5.2.7	其他规范	13

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

软件需求规约

1. 简介

1.1 目的

本软件需求规约目的是为基于多 Agent 的 C2Rust 代码翻译工具提供一个详细和完整的需求说明，旨在明确软件的外部行为，包括功能性需求和非功能性需求，以及设计约束。通过本 SRS，项目团队、利益相关者和最终用户能够理解软件系统的功能和性能要求，确保开发的产品能够满足预期目标和用户需求。

1.2 定义、首字母缩写词和缩略语

SRS: 软件需求规约 (Software Requirements Specification)，详细说明软件系统的功能和非功能需求的文档。

C2Rust: 将 C 语言代码转换为 Rust 语言代码的过程。

API: 应用程序编程接口 (Application Programming Interface)，一组预定义的函数、协议和工具，用于构建软件应用。

RAG: 检索增强生成 (Retrieval-Augmented Generation)，一种结合了检索与生成的自然语言处理技术。

AI: 人工智能 (Artificial Intelligence)，模拟人类智能行为的计算机系统。

CLI: 命令行界面 (Command-Line Interface)，用户通过文本命令与计算机交互的方式。

GUI: 图形用户界面 (Graphical User Interface)，用户通过图形化的方式 (如图标、菜单) 与计算机交互。

LML: Large Language Model，工具所调用的大语言模型，具体为 DeepSeek-V2.5。

Agent: 参与 C 到 Rust 代码自动转换过程的智能体，由 LLM 担任。每个智能体具备不同的专长和策略，共同完成复杂的代码转换任务。

CA (Computational Accuracy): 语义正确性，即测试用例通过率；

Runnable: 测试用例运行通过率；

AS-P: 将通过测试的文件中 safe API 百分比相加求平均；

AS-A: 将所有文件中 safe API 百分比相加求平均 (未通过测试的视为 0)；

CS-P: 将通过测试的文件中 safe code 百分比相加求平均；

CS-A: 将所有文件中 safe code 百分比相加求平均 (未通过测试的视为 0)；

1.3 参考资料

[Ling 2022] Michael Ling, Yijun Yu, Haitao Wu, Yuan Wang, James R. Cordy, and Ahmed E. Hassan, "In Rust We Trust - A Transpiler from Unsafe C to Safer Rust." ICSE 2022, pp. 354-355

[C2Rust] <https://github.com/immunant/c2rust/tree/master/examples>

[G-TransEval] <https://github.com/PolyEval/G-TransEval>

[Jiao 2023] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, Beijun Shen. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark, International Conference on Automated Software Engineering (ASE), Kirchberg, Luxembourg, Sept. 11-15, 2023

[xCodeEval] Mohammad Abdullah Matin Khan, M Saiful Bari, Xuan Long Do, Weishi Wang, Md Rizwan Parvez, Shafiq Joty, xCodeEval: An Execution-based Large Scale Multilingual Multitask Benchmark for Code Understanding, Generation, Translation and Retrieval, 2023, <https://arxiv.org/abs/2303.03004>

2. 整体说明

2.1 产品总体效果

本项目旨在开发一款基于多 Agent 的 C2Rust 代码翻译工具，该工具能够将 C 语言编写的代码自动转换为 Rust 语言。该工具将利用深度学习技术，特别是大模型技术，以实现高准确率的代码转换。产品的主要效果包括：提升代码的内存安全性和并发编程的安全性、减少代码转换过程中的手动干预，提高转换效

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

率、降低将现有 C 代码库迁移到 Rust 的成本和时间、提供一个用户友好的操作方式。

2.2 产品功能

- 产品整体功能为实现 C 代码到 Rust 代码的自动翻译。主要功能模块包括：
- 代码翻译模块：将 C 语言代码转换为 Rust 代码。
- 静态分析模块：对生成的 Rust 代码进行语法错误检查。
- 编译测试模块：验证 Rust 代码的语义正确性，确保与原 C 代码功能一致。
- 反馈优化模块：接收错误报告，提供修复建议，并对代码进行优化。

2.3 用户特征

- 目标用户为具备基本的编程知识、能够理解和操作代码转换工具，且有代码自动转换需求的群体，如：
- 嵌入式系统开发者：需要将 C 代码迁移到 Rust 以提升系统性能和安全性。
- 企业开发者：希望提升现有代码库的安全性和可靠性。
- 研究人员和学生：对 Rust 语言和代码转换技术感兴趣，需要学习和研究工具。

2.4 约束

- 技术约束：使用 Python 语言进行主体代码实现，部分自动化测试脚本和评价指标的计算可使用 Shell 语言。
- 时间约束：项目从 2024 年 10 月 16 日起至 2025 年 1 月 16 日止，为期 3 个月。
- 资源约束：项目团队由四名研究生组成，需要合理分配人力资源。

2.5 假设与依赖关系

- 项目假设为：Rust 语言和 C 语言的语法和语义差异可以被有效识别和转换；深度学习模型能够准确学习和模拟 C 到 Rust 的代码转换规则；用户具备基本的编程知识，能够理解和操作转换工具。
- 项目依赖于开源代码分析工具 tree_sitter 和开源大模型 DeepSeek-V2.5。

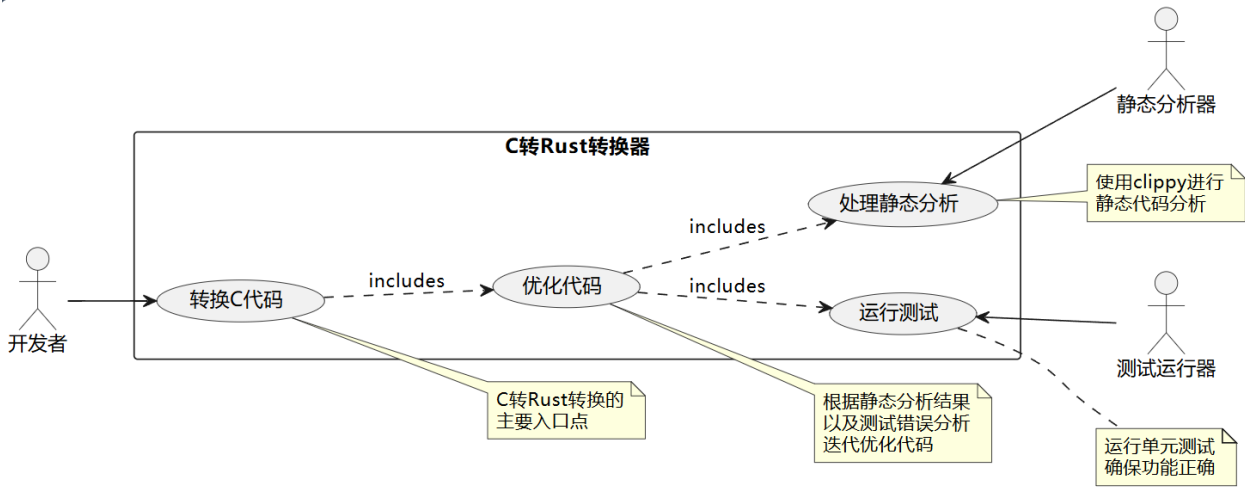
2.6 需求子集

需求子集将根据产品的功能模块进行划分，具体包括代码翻译需求、静态分析需求、编译测试需求：详细描述编译测试模块的功能和性能要求、反馈优化需求和用户操作需求。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

3. 功能需求

3.1 Use case 图



3.2 Use case1: 转换 C 代码-规约

3.2.1 概述

UC1 **转换 C 代码** 是 C 转 Rust 转换器的主要入口点，负责将开发者提供的 C 代码转换为 Rust 代码。该用例涵盖代码解析、语法转换、语义分析等步骤。

3.2.2 参与者

开发者 (Actor)：提供要转换的 C 代码，并接收转换后的 Rust 代码。

C 转 Rust 转换器 (系统)：执行代码转换的主要功能。

3.2.3 前置条件

开发者已安装并配置好 C 转 Rust 转换器。

开发者具备提供有效的 C 代码的能力。

3.2.4 事件流

3.2.4.1 主事件流

开发者 提供待转换的 C 代码。

C 转 Rust 转换器 接收并解析 C 代码。

转换器 进行语法转换，将 C 代码结构映射到 Rust 语法。

转换器 进行语义分析，确保转换后的代码在逻辑上与原始代码一致。

转换器 生成初步的 Rust 代码。

完成转换后，**转换器** 将初步的 Rust 代码传递给 **处理静态分析** (UC3) 和 **运行测试** (UC4) 进行反馈。

根据 UC3 和 UC4 的反馈，决定是否需要执行 **优化代码** (UC2)。

3.2.4.2 备用事件流

• A1: C 代码解析失败

原因：C 代码存在语法错误或不支持的语法结构。

处理：转换器返回错误信息给开发者，停止转换过程。

• A2: 语义分析不一致

原因：转换过程中发现语义上的不一致，无法确保代码逻辑保持一致。

处理：转换器返回警告信息，提示开发者手动检查相关部分。

3.2.5 后置条件

如果转换成功，**开发者** 拥有等效的 Rust 代码。

如果转换失败或存在警告，**开发者** 接收到相应的错误或警告信息。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

3.2.6 特殊需求

转换器应支持 C 语言的主要语法和常用库函数。
转换过程应记录日志，以便调试和审查。

3.3 Use case2: 优化代码-规约

3.3.1 概述

UC2 优化代码 用于根据静态分析和测试运行的反馈，对生成的 Rust 代码进行性能和质量的优化，确保代码高效且符合 Rust 的最佳实践。

3.3.2 参与者

C 转 Rust 转换器（系统）：执行代码优化的功能。

3.3.3 前置条件

UC1 转换 C 代码 已成功完成，并生成初步的 Rust 代码。

UC3 处理静态分析 和 UC4 运行测试 已完成，并提供优化需求的反馈。

3.3.4 事件流

3.3.4.1 主事件流

转换器 接收 UC3 和 UC4 的反馈信息。
优化器 分析反馈，识别需要优化的代码部分。
优化器 应用优化策略，如内联函数、减少内存分配、使用高效的数据结构等。
优化器 更新 Rust 代码，以反映优化后的结果。
优化器 可选择再次调用 处理静态分析（UC3）和 运行测试（UC4）以验证优化效果。
完成优化后，优化器 将优化后的 Rust 代码返回给 开发者。

3.3.4.2 备用事件流

• A1: 优化过程中发现严重性能问题

原因：初步转换的 Rust 代码存在无法自动优化的重大性能瓶颈。
处理：优化器返回警告信息，建议开发者手动优化相关代码部分。

3.3.5 后置条件

Rust 代码经过优化后，性能提升，符合 Rust 的最佳实践。
如果存在无法自动优化的部分，开发者 接收到相应的警告信息。

3.3.6 特殊需求

优化器应支持 Rust 的主要优化技术。
保证优化过程不改变代码的原有逻辑和功能。

3.4 Use case3: 处理静态分析-规约

3.4.1 概述

UC3 处理静态分析 负责对优化前后的 Rust 代码进行静态代码分析，确保代码质量和安全性，通过自动化工具检测潜在的代码缺陷和优化机会。

3.4.2 参与者

静态分析器（Actor）：执行具体的静态代码分析任务。
C 转 Rust 转换器（系统）：协调静态分析的执行。

3.4.3 前置条件

UC1 转换 C 代码 已成功完成，并生成初步的 Rust 代码。
Clippy 静态分析工具已集成到转换器中。

3.4.4 事件流

3.4.4.1 主事件流

转换器 将初步生成的 Rust 代码传递给 Clippy 进行静态分析。
Clippy 执行静态代码分析，检查代码中的潜在问题和改进建议。
Clippy 生成分析报告，包括警告和建议。
转换器 处理分析报告，决定是否需要执行 优化代码（UC2）。
如果需要，转换器 将优化需求反馈给 UC2 优化代码 进行调整。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

3.4.4.2 备用事件流

- **A1: 静态分析发现严重代码缺陷**
原因: 代码中存在可能导致运行时错误的严重缺陷。
处理: 转换器停止转换过程, 返回错误信息给 **开发者**。
- **A2: 静态分析工具故障**
原因: Clippy 工具出现运行错误或崩溃。
处理: 转换器记录错误日志, 并通知 **开发者** 进行手动检查。

3.4.5 后置条件

Rust 代码质量得到保证, 潜在缺陷被识别和处理。
开发者 接收到静态分析的报告和优化建议。

3.4.6 特殊需求

Clippy 工具应保持最新, 以支持最新的 Rust 语言特性。
分析过程应快速高效, 避免显著增加转换时间。

3.5 Use case4: 运行测试-规约

3.5.1 概述

UC4 **运行测试** 负责执行生成的 Rust 代码的单元测试, 确保代码功能的正确性和稳定性。

3.5.2 参与者

测试运行器 (Actor): 执行具体的测试任务。
C 转 Rust 转换器 (系统): 协调测试的执行。

3.5.3 前置条件

UC1 **转换 C 代码** 已成功完成, 并生成初步的 Rust 代码。
Rust 测试框架 (如 cargo test) 已集成到转换器中。

3.5.4 事件流

3.5.4.1 主事件流

转换器 准备测试环境, 确保所有测试依赖项已配置。
测试运行器 执行 Rust 代码的单元测试。
测试运行器 收集测试结果, 包括通过的测试、失败的测试及其详细信息。
测试运行器 将测试结果返回给 **转换器**。
转换器 评估测试结果, 决定是否需要执行 **优化代码** (UC2)。
如果存在测试失败, **转换器** 将失败信息反馈给 **开发者** 进行修复。

3.5.4.2 备用事件流

- **A1: 测试运行器遇到运行错误**
原因: 测试框架配置错误或代码存在运行时错误, 导致测试无法正常执行。
处理: 转换器记录错误日志, 并通知 **开发者** 进行手动检查和修复。
- **A2: 测试覆盖率不足**
原因: 生成的 Rust 代码缺乏足够的测试覆盖, 不能完全验证代码功能。
处理: 转换器提示 **开发者** 增加测试用例, 提升测试覆盖率。

3.5.5 后置条件

Rust 代码通过所有单元测试, 功能正确无误。
如果存在测试失败, **开发者** 需要进行相应的修复和重新测试。

3.5.6 特殊需求

测试框架应支持自动化执行和结果报告。
测试过程应确保隔离性, 避免外部因素影响测试结果。

4. 非功能需求

4.1 易用性

工具应提供清晰的反馈信息和帮助文档, 以便用户在遇到问题时能够快速找到解决方案。如有 GUI 界面, 应符合 Microsoft 的 GUI (Graphical User Interface) 标准。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

对于具有基本编程知识的普通用户，预计需要不超过 2 小时的培训时间来熟悉工具的基本操作。

对于具有 C 和 Rust 语言经验的高级用户，预计需要不超过 1 小时的培训时间来掌握工具的高级功能和自定义选项。

4.2 可靠性

4.2.1 错误界定

小错误：部分函数级别的代码翻译中断或结果丢失，而其他函数级别的代码能继续执行翻译。

大错误：部分文件级别的代码翻译中断或结果丢失，而其他文件级别的代码能继续执行翻译。

严重错误：翻译结果丢失、系统运行中途崩溃。

4.2.2 可靠性需求

系统应通过容错手段，确保当部分函数或文件级别的代码因各种原因翻译中断后，剩余代码能继续执行翻译流程。

系统应保证 99.9%的可用性，即年停机时间不超过 8.76 小时。对于小错误与大错误，平均发生间隔应分别达到 10h、15h。对于严重错误，平均发生间隔应达到 1000h。

4.3 性能

本系统的性能指标为单个函数的翻译速度。该指标平均值应达到 60s，最大不超过 300s。

4.4 可支持性

4.4.1 编码标准与命名约定

本系统编码标准和变量、函数、类和模块的命名规范均采用 Python 的 PEP 8。

4.4.2 库和框架

优先使用广泛支持的标准类库，以确保代码的可移植性和可维护性。使用第三方库时，必须选择活跃维护且社区支持良好的库，并定期检查其更新和安全性。

4.4.3 维护访问权

系统提供完整的系统文档，包括用户手册等，以便维护团队能够理解和修改系统。使用版本控制系统 Git 来管理源代码，使用 GitHub 仓库进行远程同步与协作，确保代码的变更历史和分支管理。

4.4.4 维护实用程序

实现详细的日志记录机制，以便在系统出现问题时能够追踪和诊断问题。

4.5 设计约束

4.5.1 软件语言与开发环境

必须使用 Python 语言进行主体代码的实现，以利用其丰富的库支持和社区资源。部分自动化测试脚本和评价指标的计算必须使用 Shell 语言，以便于跨平台的自动化部署和执行。

4.5.2 软件流程需求

项目采用敏捷开发的 Scrum 方法，以快速响应需求变化和技术挑战。分多个迭代进行，每个迭代周期内交付可工作的增量功能，并根据测试数据结果的反馈快速调整和优化。

4.5.3 开发工具

使用集成开发环境进行开发，使用 Git 进行版本控制，以确保代码的高效管理和协作。

4.5.4 框架及设计约束

软件采用模块化设计，最大限度地降低耦合，以便于各个功能模块的独立开发和测试。

Agent 使用 DeepSeek-V2.5 大模型，静态分析模块使用 Rust 官方维护的 Lint 静态分析工具 Clippy，代码分析使用开源工具 tree_sitter。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

5. 其它产品需求

5.1 接口需求

5.1.1 用户界面

5.1.1.1 界面内容

软件将提供一个基于 Web 的用户界面（Web UI），允许用户通过浏览器进行交互。Web UI 设计现代、响应迅速，支持多种功能模块以满足用户在代码转换和分析过程中的各种需求。

- 主页
功能：展示应用程序的简介、功能概述以及使用指南。
元素：欢迎信息，功能模块导航链接，简短的使用教程
- 文件上传与管理
功能：允许用户上传 C 代码文件或指定 C 代码目录，管理已上传的文件。
元素：文件拖放区域，文件列表显示，文件删除与重命名功能
- 转换配置
功能：用户可以配置转换参数，如输入目录、输出目录、转换选项等。
元素：表单输入字段（输入目录、输出目录），转换选项复选框或下拉菜单，提交按钮
- 转换日志
功能：实时显示转换过程的进度和详细日志信息。
元素：日志窗口（实时更新）
- 转换结果展示
功能：展示转换后的 Rust 代码，允许用户下载或复制。
元素：代码高亮显示区域

5.1.1.2 技术规格

- 前端技术：HTML5, CSS3, JavaScript（使用 React.js 或 Vue.js 框架）
- 后端技术：与前端交互的 RESTful API
- 通信协议：HTTPS
- 端口：标准的 Web 服务端口（如 443）
- 兼容性：支持主流浏览器（Chrome, Firefox, Edge, Safari）

5.1.1.3 用户交互流程

- 用户访问主页，查看应用简介。
- 用户导航到文件上传模块，上传 C 代码文件或指定目录。
- 用户配置转换参数，选择所需的转换选项。
- 用户启动转换过程，实时查看进度和日志。
- 转换完成后，用户查看转换结果，下载或复制 Rust 代码。

5.1.1.4 错误处理

- 无效输入：在用户输入无效路径或格式错误时，Web UI 将显示具体的错误提示。
- 上传失败：在文件上传过程中发生错误时，显示错误信息并允许用户重试。
- 转换失败：在转换过程中遇到问题时，显示详细的错误日志，并建议用户检查代码或联系支持。

5.1.2 硬件接口

- 本软件对硬件接口的依赖较少。它依赖于标准的计算机硬件资源，如处理器、内存和存储设备，以执行代码转换和静态分析任务。
- 处理器：至少双核处理器，推荐四核或以上，以提高多任务处理能力。
- 内存：至少 4 GB RAM，推荐 8 GB 或更多，以支持大规模代码库的处理。
- 存储：需要足够的磁盘空间来存储输入的 C 代码、输出的 Rust 代码及生成的日志文件。具体需求取决于项目规模，建议至少 100 MB 的可用空间。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

文件系统：软件需要访问和操作文件系统，以读取 C 代码文件、写入 Rust 代码文件和日志文件。需要适当的文件权限。

5.1.3 软件接口

5.1.3.1 外部软件组件：

- Clippy (Rust 的静态分析工具)：
命令: clippy-driver 或 cargo clippy
接口描述: 通过后端子进程调用 Clippy 进行 Rust 代码的静态分析。捕获并解析 Clippy 的输出以识别代码中的问题。
协议与端口: 基于本地命令行调用，无需网络协议或端口。
- Cargo (Rust 的包管理工具)：
命令: cargo test
接口描述: 通过后端子进程调用 Cargo 执行 Rust 测试。捕获并解析测试结果以确定测试是否通过。
协议与端口: 基于本地命令行调用，无需网络协议或端口。
- LLM Agents:
组件名称: api_agent, syntax_agent, feedback_agent, optimize_agent
接口描述: 这些 agents 用于生成和优化代码转换和修复建议。它们的具体实现依赖于本地模型或远程 API 接口。
协议与端口:
本地实现: 通过本地部署大模型并调用进行通信。
远程 API: 依赖于远程 API (如 HTTP/HTTPS)，需要指定 API_KEY。

5.1.3.2 内部软件组件

软件内部的模块和函数相互协作，包括但不限于：
预处理模块：处理文件编码、字符串规范化等。
转换模块：执行 C 到 Rust 的代码转换。
分析模块：进行静态和动态分析，捕获和处理 Clippy 和 Cargo 的输出。
日志模块：记录程序运行过程中的信息、警告和错误。
Web 服务器模块：处理前端 Web UI 的请求与响应。

5.1.4 通信接口

软件在大模型 api 调用时需要进行远程通信：
协议: HTTP 或 HTTPS，具体取决于大模型。
端口: 根据 API 服务的配置，常见端口为 80 (HTTP) 或 443 (HTTPS)。
逻辑地址: API Agents 可能托管在本地服务器或第三方云服务上。需要明确 API 的 URL 或 IP 地址。
数据格式: 基于 REST 的 JSON 请求和响应，具体取决于 API 文档。

5.2 适用的标准

5.2.1 法律法规

《中华人民共和国知识产权法》：项目中使用的开源代码和第三方库必须严格遵守相应的知识产权法律，确保合法使用和授权。
《中华人民共和国网络安全法》：处理个人数据时，必须符合相关隐私保护法律要求，确保用户数据的安全和隐私。

5.2.2 质量标准

ISO/IEC 25010:2011：系统和软件质量要求，包括功能性、性能效率、兼容性、可用性、可靠性、安全性和可维护性等方面。
IEEE/ISO 24765:2017：软件工程 - 软件产品的质量要求和评价。

C2Rust	Version: <1.2>
软件需求规约	Date: <12/11/2024>

5.2.3 易用性标准

IBM CUA (Common User Access): IBM 的通用用户访问标准, 提供用户界面设计指南, 以提高软件的易用性。

Microsoft GUI (Graphical User Interface) Standards: 微软图形用户界面标准, 提供一致性和可访问性的指导原则。

5.2.4 互操作性标准

Rust API Guidelines: Rust 语言的 API 设计指南, 确保生成的 Rust 代码与 Rust 生态系统中的其他库和工具兼容。

C Language Standards (ISO/IEC 9899): C 语言的国际标准, 确保源 C 代码的兼容性和互操作性。

5.2.5 国际化标准

Unicode Standard: 软件必须支持 Unicode 字符集, 以确保全球用户能够使用自己的语言 and 特殊字符。

ISO/IEC 19790: 国际化软件的测试和质量标准, 确保软件能够适应不同的语言和文化环境。

5.2.6 安全性标准

OWASP Top Ten Project: 关注网络安全风险, 提供软件安全开发的最佳实践。

ISO/IEC 27000 series: 信息安全管理标准, 包括安全策略、风险评估和控制措施。

5.2.7 其他规范

Rust 官方编码规范: 遵循 Rust 官方的编码规范和最佳实践, 确保生成的 Rust 代码的质量和一致性。